Applying DI

Agenda

OBJECT TECHNOLOGIES™

1. Usefulness and approach
2. IOC containers
3. Steps for DI
4. Types of DI
5. Bean scopes
6. Autowiring

1. Usefulness and approach

OBJECT™ TECHNOLOGIES

- Dependency injection decreases coupling between classes and its dependency.
- Due to increase in decoupling, the reusability of the code is increased.
- The ease of testing is another benefit when using dependency injection.
- Basic approach:
  - Define interface or abstract class
  - Make concrete implementations
  - Declare concrete object in bean definition file
  - Instantiate container from bean definition file.
  - Get bean instance(s)

2. IOC containers

OBJECT TECHNOLOGIES

- The IoC container is responsible to instantiate, configure and assemble the objects.
- The IoC container gets informations from the XML file.
- Types of IoC containers :
- BeanFactory
  - This is the simplest container providing the basic support for DI.
  - This container reads the configuration metadata from an XML file
- ApplicationContext
  - The Application Context is Spring's advanced container.
  - Similar to BeanFactory, it can load bean definitions, wire beans together, and dispense beans upon request.
  - The most commonly used ApplicationContext implementations are FileSystemXmlApplicationContext, ClassPathXmlApplicationContext and WebXmlApplicationContext.

3. Steps for DI

- Add the spring framework jar files eiter physically or using the build tool like Maven.
- Create POJO class. It may have several properties or attributes of different nature like built-in type, collection type or reference type
- In the bean definition XML file, <beans> is a root tag. <bean> tag defines beans where <element> tag defines properties which can be assigned with a constant value ot referred from some another bean.
- Instantiate the container which should be done only once as it is performance heavy. If the bean definition XML file is on classpath, we can use ClassPathXMLApplicationContext
- Get the instance of bean. By default, calling getBean() on the same name multiple times returns the same instance which can be used for calling business logic methods.

4. Types of DI

- The two major types of Spring Framework Dependency Injection are:
- Constructor-Based Dependency Injection
  - When the class constructor is invoked by the container with the number of arguments each having a dependency on other class.
- Setter-Based Dependency Injection
  - When the container creates bean instance using default constructor and the for initializing bean properties all the setter methods are used

5. Bean scopes

OBJECT™ TECHNOLOGIES

- In bean definition, we can control the scope of the objects created from a particular bean definition.
- Spring Framework supports exactly five scopes (of which three are available only if you are using a web-aware ApplicationContext).
  - Singleton: a single object instance per Spring IoC container.
  - prototype: any number of object instances.
  - request: a single bean definition to the lifecycle of a single HTTP request
  - session: a single bean definition to the lifecycle of a HTTP session

6. Autowiring

- Spring framework provides autowiring features too where we don't need to provide bean injection details explicitly.
- The @Autowired annotation provides more fine-grained control over where and how autowiring should be accomplished.
- Modes of autowiring :
  - Default autowiring mode is 'no'. It means no autowiring.
  - Autowiring 'byName' : Spring looks up the configuration file for a matching bean name. If found, this bean is injected in the property.
  - Autowiring 'byType' : It searches property's class type in configuration file. It injects the property, if such bean is found, otherwise an error is raised.
-

6. Autowiring

- Autowiring ‘constructor’ : Autowiring by constructor is similar to byType but it applies to constructor arguments. It will look for the class type of constructor arguments, and then do an autowire byType
- Autowiring by autodetect : First, it will look for valid constructor with arguments. If it is found then the constructor mode is chosen. If there is no constructor defined in a bean, the autowire byType mode is chosen. This is possible by using @Autowired annotation